南開大學

信息隐藏技术课程实验报告

实验一：可视密码学

学　　院　密码与网络空间安全学院

专　　业　信息安全、法学双学位班

学　　号　2313815

姓　　名　段俊宇

班　　级　信息安全、法学双学位班

一、实验要求

1、 实现二值图像的(2,2)的可视密钥共享方案。

2、 实现灰度图像和彩色图像的(2,2)的可视密钥共享方案。

3、 (t,n)方案。

4、 叠像术。

二、实验原理

Shamir(k,t)门限方案：在 n 个参与者之间分享一个秘密，k 个或 k 个以上参与者可以恢复秘密，而少于 k 个参与者则无法恢复秘密。

二值信息分存方案：将原始图像的每个像素被两个子块所替代，每个子块由 2×2 的黑、白像素构成，这样就生成了两幅膨胀的图形，两幅图像叠加得到了放大 4 倍且对比度下降的原始图像。

半色调图像：通过特殊加工技术，使用大小不同的微小网点表现由浅到深或由淡到浓的色调变化。该技术利用人眼视觉的低通滤波特性，仅用单一颜色的油墨，就能在只能呈现二值的介质上再现出具有连续明暗层次的视觉效果。

误差扩散法：先阈值量化图像像素，然后将图像量化过程中产生的误差分配到周边像素。利用这种方法生成的半色调图像能够比较忠实地反应图像的灰度层次过渡。灰度图像每个像素点用 8bit 表示，范围在 0-255，因此如果像素值小于 127 则输出 0（黑色）；大于 127 则输出 1（白色）。在扩散时，给右方、右下方、正下方、左下方四个像素点分别加上权值为 7/16、1/16、5/16、3/16 的误差。

灰度图像可视密码学实现：在二值信息可视密码实现基础上，使用误差扩散法实现。

彩色图像可视密码学实现：彩色图像每个像素由红、绿、蓝三个分量构成，存储时对应一个三维矩阵。可以将彩色图像看做三个分量上的三张图片，然后对每张图片按照半色调处理，R、G、B 分量存到两张子图（分享图）中，最后将得到的子图合并可以得到两张彩色子图。

(t,n)门限方案：该方案将秘密图像分解成 n 张分享图，使得任意 t 张或更多分享图叠加能恢复原图，少于 t 张则得不到任何信息。在该方案的实

现过程中，原秘密图像的每个像素被扩展为 m 个子像素，并通过一定设计将原始秘密图像分割为 n 个子图像，每个子图像由 t 个像素的组合构成，当 t 个子图像叠加时，秘密像素的对应位置会因黑色子像素的叠加而显现。实现该方案所用的参考文献为“董晨、张皓宇、季姝廷、李磊：《基于异或的外形比例不失真(k, n)视觉密码方案》”。

三、实验环境

Matlab_R2025b，Image Processing Toolbox

小组成员：段俊宇（2313815）、陈展（2313485）、李相儒（23130591）

四、实验过程

在实现该代码时，我将三种图像信息的分存和误差扩散法封装成了函数，如下是图像的起始处理及图像处理的选择：

```
function Visual_Cryptography()
    % 读取图像
    img = imread("1.png");

    % 选择模式
    mode = input('请选择: 1-二值, 2-灰度, 3-彩色: ');

    switch mode
        case 1
            processBinary(img);
        case 2
            processGray(img);
        case 3
            processColor(img);
    end
end
```

1. 二值图像的(2,2)的可视密钥共享方案

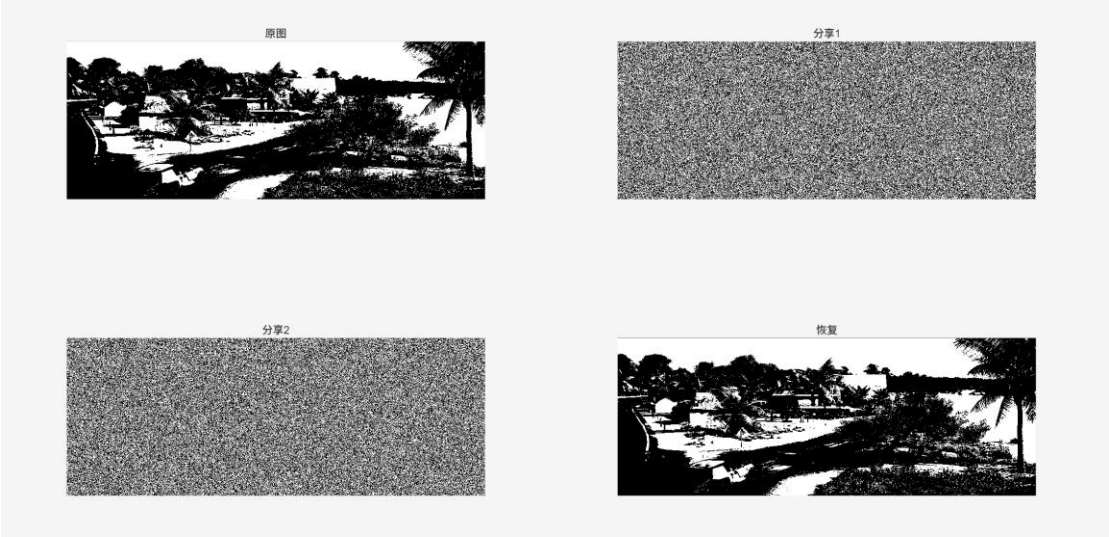
如下是 Matlab 代码，使用注释进行解释：

```
% 二值图像处理
function processBinary(img)
    % 如果是彩色图像转换为二值图像
    if size(img,3)==3
        img = rgb2gray(img);
    end
    img = imbinarize(img);
    % 获取原始图像的长宽
```

```
    [h,w] = size(img);
    % 初始化分享图
    s1 = zeros(2*h,2*w);
    s2 = zeros(2*h,2*w);
    % 预定义像素块模式，第一种模式为白黑黑白，第二种模式为黑白白黑
    patterns = {
        [1 0;0 1],
        [0 1;1 0]
    };

    % 编码
    for i=1:h
        for j=1:w
            r = randi(2);
            p = patterns{r};
            % 如果像素为白色
            if img(i,j)==1
                s1(2*i-1:2*i,2*j-1:2*j) = p;
                s2(2*i-1:2*i,2*j-1:2*j) = 1-p;
            % 像素为黑色
            else
                s1(2*i-1:2*i,2*j-1:2*j) = p;
                s2(2*i-1:2*i,2*j-1:2*j) = p;
            end
        end
    end

    % 恢复后的图像
    recover = xor(s1, s2);

    % 显示结果
    figure;
    subplot(221); imshow(img); title('原图');
    subplot(222); imshow(s1); title('分享 1');
    subplot(223); imshow(s2); title('分享 2');
    subplot(224); imshow(recover); title('恢复');

    imwrite(s1, "Binary Visual Secret Sharing_1.png");
    imwrite(s2, "Binary Visual Secret Sharing_2.png");
    imwrite(recover, "Binary Recovery.png");
end
```

在测试代码时我使用下面的彩色图像：

运行上述代码，得到如下的对比图：



从图中可以看出，原先的彩色图像被转换成了二值图像，两张分享图都毫无意义，但是通过对两张图片的每个像素点进行异或操作可以得到恢复后的图片，和原图几乎一模一样，说明该方案实现成功！

2. 灰度图像的(2,2)的可视密钥共享方案

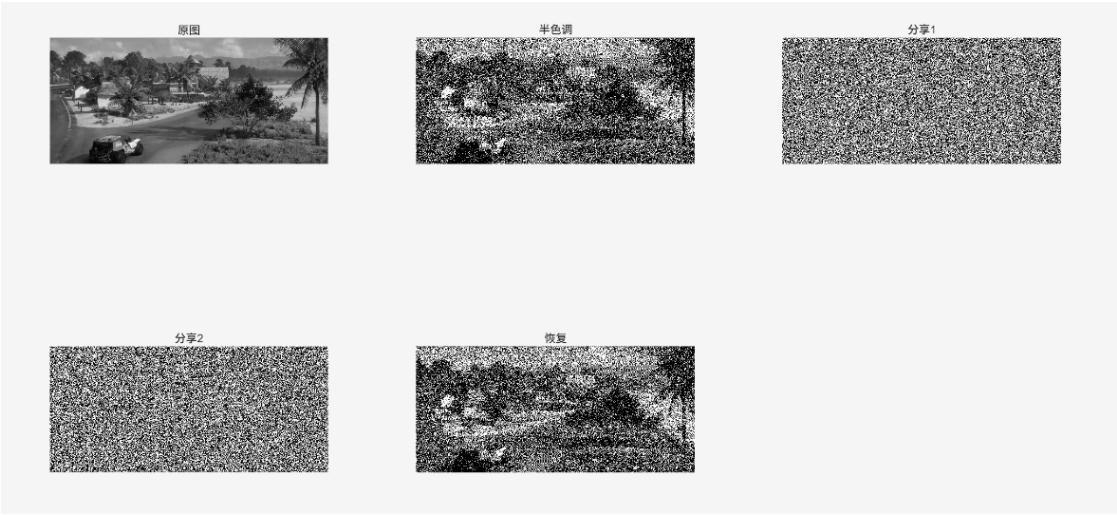
代码如下所示，使用注释进行解释：

```
% 灰度图像处理（使用误差扩散半色调）
function processGray(img)
    % 如果是彩色图像转换为二值图像
    if size(img,3)==3
        img = rgb2gray(img);
    end

    % 使用误差扩散半色调
    bw = errorDiffusionHalftone(img);
```

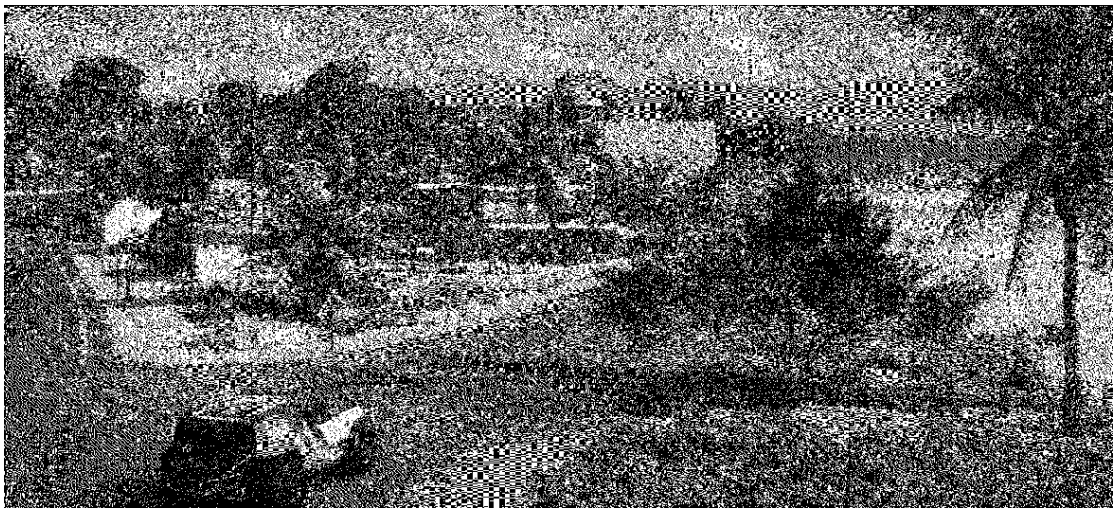
```

    [h,w] = size(bw);
    s1 = zeros(2*h,2*w);
    s2 = zeros(2*h,2*w);
    patterns = {
        [1 0;0 1],
        [0 1;1 0]
    };

    for i=1:h
        for j=1:w
            r = randi(2);
            p = patterns{r};
            if bw(i,j)==1
                s1(2*i-1:2*i,2*j-1:2*j) = p;
                s2(2*i-1:2*i,2*j-1:2*j) = 1-p;
            else
                s1(2*i-1:2*i,2*j-1:2*j) = p;
                s2(2*i-1:2*i,2*j-1:2*j) = p;
            end
        end
    end

    recover = xor(s1,s2);

    figure;
    subplot(231); imshow(img); title('原图');
    subplot(232); imshow(bw); title('半色调');
    subplot(233); imshow(s1); title('分享 1');
    subplot(234); imshow(s2); title('分享 2');
    subplot(235); imshow(recover); title('恢复');

    imwrite(bw, "Halftone.png");
    imwrite(s1, "Grayscale Visual Secret Sharing_1.png");
    imwrite(s2, "Grayscale Visual Secret Sharing_2.png");
    imwrite(recover, "Grayscale Recovery.png");
end

% Floyd-Steinberg 误差扩散半色调
function bw = errorDiffusionHalftone(img)
    % 转换为 double 便于计算
    img = double(img);
    [h,w] = size(img);
    bw = zeros(h,w);
```

```

    % 逐像素处理
    for i=1:h
        for j=1:w
            old = img(i,j);
            % 判断原像素是否大于 127 并将逻辑值存入数组
            if old > 127
                new = 255;
            else
                new = 0;
            end
            bw(i,j) = new > 128;

            % 计算误差
            error = old - new;

            % 扩散误差到相邻像素
            if j < w
                img(i, j+1) = img(i, j+1) + error * 7/16;
            end
            if i < h
                if j > 1
                    img(i+1, j-1) = img(i+1, j-1) + error * 3/16;
                end
                img(i+1, j) = img(i+1, j) + error * 5/16;
                if j < w
                    img(i+1, j+1) = img(i+1, j+1) + error * 1/16;
                end
            end
        end
    end
end
```

对代码进行测试，结果如下所示：



从图中可以发现，最后恢复的图像与半色调图像基本相同，两张分享图同样毫无意义，但是组合可以得到原图，说明实现成功！此外，当我单独打开被恢复的图像时，放到 word 文档中如下所示：



之后我将该图像的大小调整至与原图相同，得到了如下图像

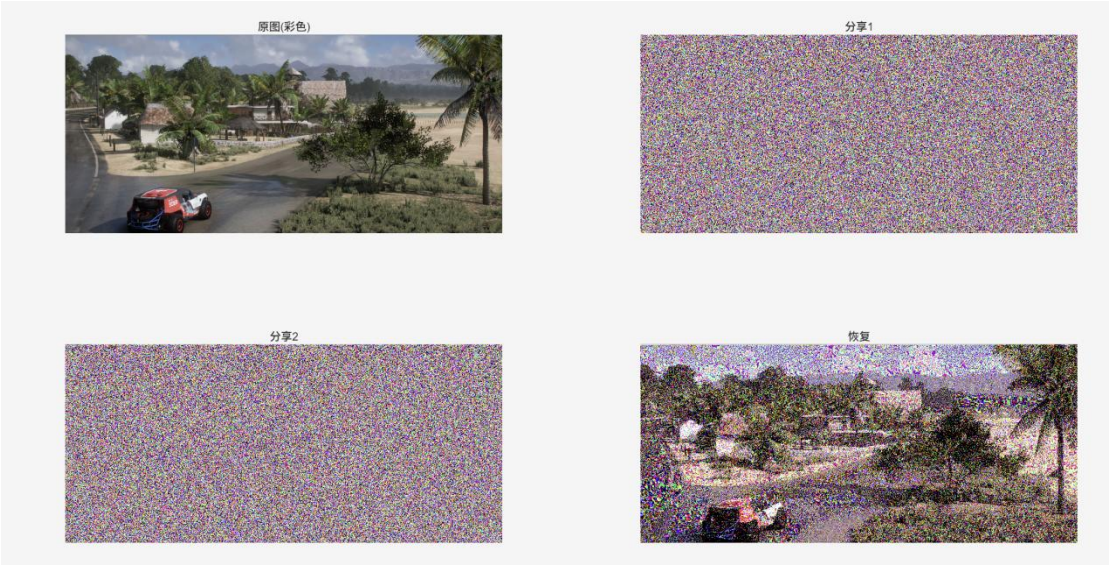
询问 AI 后得知这是因为半色调图像由很多网点构成，被恢复的图像保留了更多的网点颗粒；该图像被缩小后破坏了原本的结构，而后再放大时会更像连续调图像，因此显得更加平滑。

3. 彩色图像的(2,2)的可视密钥共享方案

代码如下所示，使用注释进行解释：

```
% 彩色图像处理
function processColor(img)
    if size(img,3)~=3, error('需要彩色图'); end

    % 分离 RGB 通道
    R = img(:,:,1); G = img(:,:,2); B = img(:,:,3);

    % 分别处理每个通道（使用误差扩散）
    [s1R,s2R] = processColorChannel(R);
    [s1G,s2G] = processColorChannel(G);
    [s1B,s2B] = processColorChannel(B);

    % 合成彩色分享图
    s1 = cat(3, uint8(s1R*255), uint8(s1G*255), uint8(s1B*255));
    s2 = cat(3, uint8(s2R*255), uint8(s2G*255), uint8(s2B*255));

    % 叠加恢复
    restoredR = xor(s1R, s2R);
    restoredG = xor(s1G, s2G);
    restoredB = xor(s1B, s2B);

    % 转换为可显示的格式
    recover = cat(3, uint8(restoredR*255), ...
                     uint8(restoredG*255), ...
                     uint8(restoredB*255));

    figure;
    subplot(221); imshow(img); title('原图(彩色)');
    subplot(222); imshow(s1); title('分享 1');
    subplot(223); imshow(s2); title('分享 2');
    subplot(224); imshow(recover); title('恢复');

    imwrite(s1, "Color Visual Secret Sharing_1.png");
    imwrite(s2, "Color Visual Secret Sharing_2.png");
    imwrite(recover, "Color Recovery.png");
end
```

对代码进行测试，结果如下所示：



从图中可以发现，最后恢复的图像与原始图像近似相同，将其缩小后会更加平滑。两张分享图同样毫无意义，但是组合可以得到原图，说明实现成功！

4.(t,n)门限方案

该方案代码由小组成员完成，代码如下所示，原始代码文件在threshold_secret_share_binary.m中。

```
function sub_images = threshold_secret_share_binary(k, n, image_path)
    % t: 恢复秘密图像所需的最小子图像数
    % n: 生成的子图像总数
    % image_path: 二值秘密图像的文件路径
    % sub_images: 返回值，用于存储所有生成份额的单元格数组
    % 1、处理输入图像：支持路径或矩阵，进行逻辑取反，黑色 1，白色 0
    if ischar(image_path) || isstring(image_path)
        binary_image = imread(image_path) ~= 0;
    else
        binary_image = logical(image_path);
    end
    binary_image = ~binary_image;
    % 2、获取秘密图像的尺寸，初始化 n 个子图像存储空间
    [height, width] = size(binary_image);
    sub_images = cell(1, n);
    % 3、初始化加密矩阵
    C0 = [];
    C1 = [];
    for p = 0:k
```

```
        % 如果 p 为偶数，添加新列至 C0
        if mod(p, 2) == 0
            % get_q_ones_vectors(q, n): 返回单列中存在 k 个 1 的所有列向量，共 C_n_q 个
            % howmany_ones_need(p, k, n): 如果 2p <= k，输出 q = p；否则输出 q = n + p - k
            q = howmany_ones_need(p, k, n);
            C0 = [C0, get_q_ones_vectors(q, n)];
        end
        % 如果 p 为奇数，添加新列至 C1
        if mod(p, 2) == 1
            q = howmany_ones_need(p, k, n);
            C1 = [C1, get_q_ones_vectors(q, n)];
        end
    end
    % 4、循环处理多余列直至结束
    % 初始化可用资源列信息
    C0_use = ones(1, size(C0, 2));
    C1_use = ones(1, size(C1, 2));
    % 如果存在列未被使用
    while any(C0_use) || any(C1_use)
        if any(C1_use)
            % 提取可使用列，并对其可用标记清零
            C1_new = C1(:, logical(C1_use));
            C1_use(logical(C1_use)) = 0;
            % get_odd_weights(A, k, who): 对一矩阵任取 k 行，若存在这 k 行中某列包含奇(偶)数个 1，记录 k 行下该列包含 1 的个数 r，返回所有可能的 r
            rs = get_odd_weights(C1_new, k, 1);
            % 对每个多余量 r，添加多余列至 C0
            for r_i = 1:length(rs)
                q = howmany_ones_need(rs(r_i), k, n);
                add = get_q_ones_vectors(q, n);
                C0 = [C0, add];
                C0_use = [C0_use, ones(1, size(add, 2))];
            end
        end
        if any(C0_use)
            C0_new = C0(:, logical(C0_use));
            C0_use(logical(C0_use)) = 0;
            rs = get_odd_weights(C0_new, k, 0);
            for r_i = 1:length(rs)
                q = howmany_ones_need(rs(r_i), k, n);
                add = get_q_ones_vectors(q, n);
                C1 = [C1, add];
                C1_use = [C1_use, ones(1, size(add, 2))];
```

```
            end
        end
    end
    % 5、利用加密矩阵得到 n 个共享份
    m = size(C0, 2);
    get_n = ceil(sqrt(m));
    e_b = zeros(1, m + 1);
    % 按 get_n 行对矩阵 binary_image 进行切割并列优先拼接
    num_blocks = ceil(height / get_n);
    blocks_cell = cell(num_blocks, 1);
    for i = 1:num_blocks
        row_range = ((i-1)*get_n + 1) : min(i*get_n, height);
        block = binary_image(row_range, :);
        blocks_cell{i} = block(:); % 列优先展开
    end
    secret_vector = vertcat(blocks_cell{:});
    % 6、根据 secret_vector 生成 n 个共享份向量 sub_vectors
    num_pixels = length(secret_vector);
    sub_vectors = cell(1, n);
    for i = 1:n
        sub_vectors{i} = zeros(1, num_pixels);
    end
    for start_idx = 1:m:num_pixels
        end_idx = min(start_idx + m - 1, num_pixels);
        current_segment = secret_vector(start_idx:end_idx);
        s = length(current_segment);
        b = sum(current_segment == 1);
        if s == m
            % 完整块处理：使用 e_b mod m < b 判断
            if mod(e_b(b+1), m) < b
                Cx = C1(:, randperm(m));
            else
                Cx = C0(:, randperm(m));
            end
        else
            % 最后一组不足 m 个像素的处理
            if b < s / 2
                Cx = C0(:, randperm(m));
            else
                Cx = C1(:, randperm(m));
            end
        end
        e_b(b+1) = e_b(b+1) + 1;
        % 将 Cx 的列值分配给 sub_vectors
```

```
        for j = 1:s
            k_pos = start_idx + j - 1;
            for i_idx = 1:n
                sub_vectors{i_idx}(k_pos) = Cx(i_idx, j);
            end
        end
    end
    % 7、根据 sub_vectors 还原 n 个 sub_images
    for i_idx = 1:n
        current_v = sub_vectors{i_idx};
        current_img = zeros(height, width);
        curr_pos = 1;
        for i = 1:num_blocks
            row_range = ((i-1)*get_n + 1) : min(i*get_n, height);
            block_h = length(row_range);
            block_size = block_h * width;
            block_data = current_v(curr_pos : curr_pos + block_size - 1);
            current_img(row_range, :) = reshape(block_data, [block_h, width]);

            curr_pos = curr_pos + block_size;
        end
        % 还原为原始图像逻辑（0 为黑，1 为白），与开头的取反操作对应
        sub_images{i_idx} = ~current_img;
    end
    % 8、可视化结果展示并保存
    % 画布：展示原秘密图像
    figure('Name', '原始秘密图像', 'NumberTitle', 'off');
    imshow(~binary_image);
    title('原始秘密图像');
    imwrite(~binary_image, 'original_secret_binary.png');
    % 画布：展示生成的 n 个密钥图（每个密钥图一个画布）
    for i = 1:n
        figure('Name', sprintf('共享份 %d', i), 'NumberTitle', 'off');
        imshow(sub_images{i});
        title(sprintf('共享份 %d', i));
        imwrite(sub_images{i}, sprintf('share_%d.png', i));
    end
    % 9、多份共享份异或还原与可视化并保存
    xor_images = cell(1, n-1);
    for many = 2:n
        selected_indices = 1:many;
        % 对选出的 many 份图像进行异或操作
        % 为了保证逻辑一致性，先将密钥图还原为“1 为黑”的状态进行异或
        temp_xor = ~sub_images{selected_indices(1)};
```

```
        for idx = 2:many
            temp_xor = xor(temp_xor, ~sub_images{selected_indices(idx)});
        end
        t = many - 1;
        % 异或结果中 1 为黑，显示时取反回“0 为黑”的状态
        xor_images{t} = ~temp_xor;
        % 画布：展示每一组异或结果
        figure('Name', sprintf('任意 %d 份异或还原结果', many), 'NumberTitle', 'off');
        imshow(xor_images{t});
        title(sprintf('任意 %d 份异或', many));
        imwrite(xor_images{t}, sprintf('xor_reconstruction_%d_shares.png', many));
    end
end
```

五、实验结论及心得体会

本次实验中我巩固了课上学习的可视密码学，在实验过程中使用 Matlab 实现了二值图像、灰度图像和彩色图像的(2,2)可视密钥分享方案以及叠像术，对于不同类型图片的像素构成有了更加深刻的理解，同时也学习了 Matlab 的语法。而(t,n)方案的代码经过小组成员合作、查阅文献，最终成功实现，让我感受到了数学在信息隐藏中的魅力。